# Sample-Size Simulation: Product-Mixture Probit Factors versus Joint-Mixture Gibbs

August 13, 2026

## 1 Purpose

This note documents the sample-size simulation used to compare two ways of fitting binary probit factor-mixture structure in high-dimensional item-response data. The simulation was designed to answer a practical question: when the latent factors have mixture structure, how much data are needed for a full joint-mixture Gibbs sampler to recover the data-generating parameters, and how does that compare with the proposed independent product-mixture fitting strategy?

The comparison uses the same binary probit data-generating model for both methods, the same post-processing alignment, and the same recovery metrics. Each Monte Carlo repetition simulates a new dataset, including new latent mixture labels, factor values, probit noise, binary responses, item intercepts, and loading matrices.

## 2 Data-Generating Model

For subject or model $i = 1, \ldots, n$ and item $j = 1, \ldots, p$, responses are generated through a latent Gaussian probit variable,

$$Z_{ij} = \alpha_j + \lambda_j^\top f_i + \epsilon_{ij}, \qquad \epsilon_{ij} \sim N(0, 1), \tag{1}$$

$$X_{ij} = \mathbf{1}\{Z_{ij} > 0\}. \tag{2}$$

The factor vector $f_i = (f_{i1}, \ldots, f_{iH})^\top$ has independent marginal mixtures,

$$f_{ih} \sim \sum_{g=1}^{G} \pi_{hg} N(\mu_{hg}, \sigma_{hg}^2), \qquad h = 1, \ldots, H. \tag{3}$$

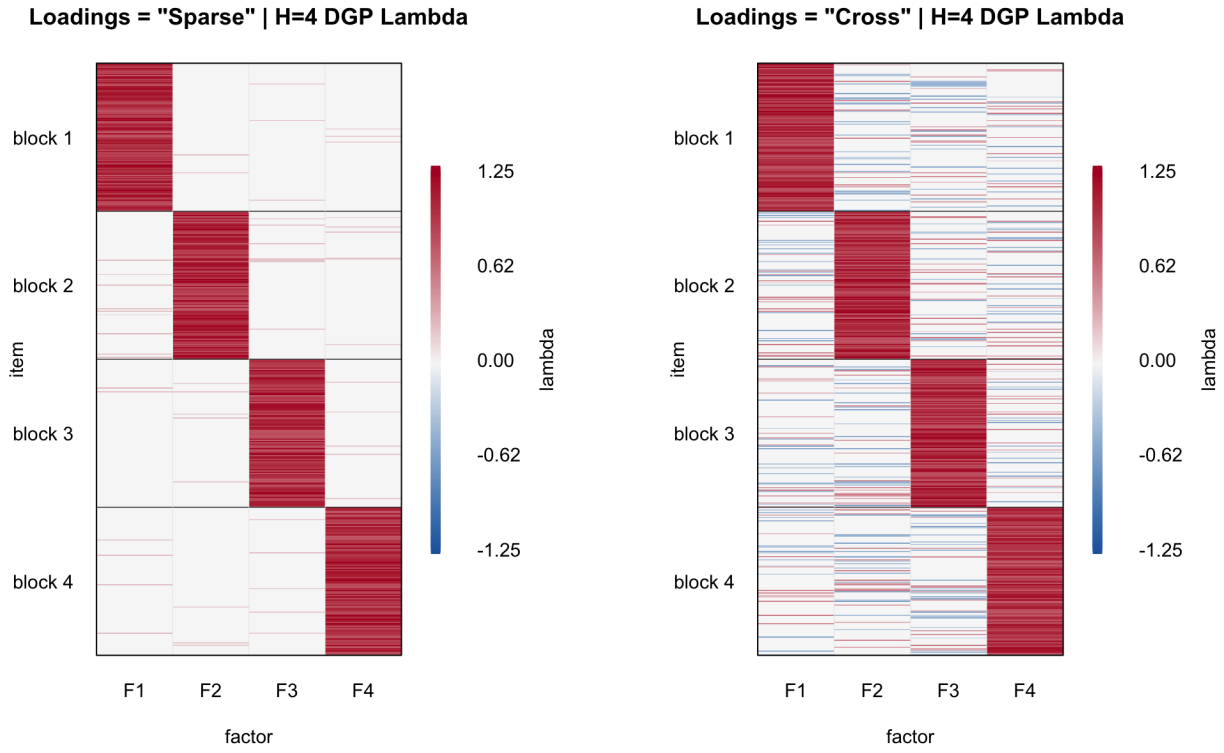
Equivalently, the full $H$-dimensional factor distribution can be written as a joint Gaussian mixture with $G^H$ profiles, where each profile corresponds to one Cartesian product of the marginal component labels. This equivalence is important: the Gibbs comparator is correctly specified as a joint mixture, while the proposed method exploits the product structure directly.

The simulation grid is:

- $H \in \{3, 4\}$ latent factors;
- $G \in \{2, 3\}$ mixture components per factor;
- $n \in \{100, 500, 1000, 2000\}$ and $p = 500$;

- 25 Monte Carlo repetitions per setting;
- IFEval-like item intercepts, with block structure plus random item-level variation;
- unequal mixture variances;
- two loading designs, labeled below as **Sparse** and **Cross**.

The Sparse loading design has one primary positive loading block per factor, with rare weak positive cross-loadings. The Cross design keeps the same primary positive block structure but adds more frequent signed cross-loadings. Figure 1 shows representative realizations of the DGP loading matrices for $H = 4$. The actual simulation redraws $\Lambda$ in each repetition; these figures are fixed-seed representative matrices generated by the same DGP code.




(a) Sparse loading design.

(b) Cross loading design.

Figure 1: Representative DGP loading matrices for $H = 4$ and $p = 500$.

# 3 Methods Compared

## 3.1 Proposed Product-Mixture Probit Factor Method

The proposed method fits the independent marginal-mixture factor model directly. The fitted model is

$$X_{ij} \mid f_i, \alpha_j, \lambda_j \sim \text{Bernoulli}\{\Phi(\alpha_j + \lambda_j^\top f_i)\}, \tag{4}$$

$$f_{ih} \sim \sum_{g=1}^{G} \pi_{hg} N(\mu_{hg}, \sigma_{hg}^2), \qquad h = 1, \ldots, H, \tag{5}$$

with independent factor coordinates. The algorithm has two stages.

**Stage 1: augmented-probit spectral pretraining.** The binary responses are augmented with latent probit variables $Z_{ij}$ drawn or updated from their truncated normal conditional distribution. The current $Z$ matrix is centered and a rank $H$ singular value decomposition is used to initialize factor scores. The resulting spectral scores are rotated so that each coordinate is well described by a one-dimensional Gaussian mixture. In this simulation the rank is treated as known and the pretraining stage runs for at most 10 augmentation/rotation iterations.

**Stage 2: MAP refinement.** Starting from the pretrained factors and loadings, the algorithm alternates among:

1. updating augmented probit variables or their conditional expectations;
2. updating item intercepts and sparse loadings through probit surrogate regressions;
3. updating factor scores under the current loading/intercept values;
4. updating each marginal mixture distribution by MAP mixture updates;
5. normalizing factor scale so the probit linear predictor is unchanged while factor mixture means and variances remain comparable across iterations.

The refinement stage also runs for at most 10 iterations. The loading penalty and mixture MAP priors are fixed across the simulation grid.

## 3.2 Joint-Mixture Factor Gibbs Comparator

The comparator is a correctly specified joint-mixture factor model. Instead of representing the factor distribution as $H$ independent $G$-component mixtures, it represents the factor vector as a single mixture with $K = G^H$ joint profiles:

$$f_i \mid c_i = k \sim N_H(\mu_k, \text{diag}(\sigma_{k1}^2, \ldots, \sigma_{kH}^2)), \tag{6}$$

$$\Pr(c_i = k) = \omega_k, \qquad k = 1, \ldots, G^H, \tag{7}$$

$$X_{ij} \mid f_i, \alpha_j, \lambda_j \sim \text{Bernoulli}\{\Phi(\alpha_j + \lambda_j^\top f_i)\}. \tag{8}$$

This model is correctly specified for the simulated data because the Cartesian product of the independent marginal mixtures induces exactly such a joint mixture.

The Gibbs sampler uses probit augmentation and cycles through:

1. sampling $Z_{ij}$ from its truncated normal full conditional given $X_{ij}$;

2. sampling or updating item intercepts and loadings conditional on $Z$ and factor scores;

3. sampling factor scores $f_i$ conditional on $Z$, loadings, intercepts, and class labels;

4. sampling joint profile labels $c_i$;

5. updating joint mixture weights, means, and diagonal variances;

6. applying the same scale normalization convention used for the proposed method.

Each Gibbs fit uses 2000 iterations, with 1000 burn-in iterations and no thinning.

# 4 Alignment and Evaluation

Factor models are unidentified up to signed permutations of the latent coordinates. To make the comparison apples-to-apples, both methods use the same post-processing strategy: choose the factor permutation and sign pattern that best aligns the estimated loading matrix with the true DGP loading matrix. The same alignment is then applied to factor scores and mixture parameters.

The plotted recovery metrics summarize:

- RMSE for item intercepts $\alpha$;
- RMSE for loadings $\Lambda$;
- RMSE for mixture means;
- RMSE for mixture variances;
- RMSE for joint-profile mixture weights, computed by vectorizing the aligned $G^H$ profile probabilities;
- flattened factor-score correlation after alignment.

Each point in the recovery plots is the mean across 25 repetitions. Error bars show plus/minus two Monte Carlo standard deviations. Runtime plots report mean log(seconds) with plus/minus two standard deviations; the raw timing field is total elapsed wall-clock time for the full method fit.

# 5 Results Summarized

Across the 32 combinations of loading design, $H$, $G$, and sample size, the proposed product mixture has lower mean RMSE than the joint Gibbs comparator for loadings, mixture means, mixture variances, and mixture weights. It also has higher mean flattened factor-score correlation in all 32 cells. At $n = 2000$, loading RMSE is close for $G = 2$ but the joint Gibbs sampler remains much less accurate for mixture means, variances, and profile weights, especially when $G = 3$ and $K = G^H$ is large.

The main interpretation is computational and statistical. Although the joint-mixture Gibbs model is correctly specified, it must estimate $G^H$ joint profiles. This creates many mixture parameters and a hard class-allocation problem in the same $n, p$ regimes where the product mixture only estimates $H$ separate one-dimensional mixtures. The product-mixture method exploits the

factorial structure, so it recovers the relevant factor axes and marginal mixture parameters with substantially less apparent sample-size burden.

Figures 2–4 show the most difficult setting, $H = 4, G = 3$, where the joint mixture has $3^4 = 81$ profiles. The full set of selected plots for all settings is stored in the repository under the selected sample-size plot directory.

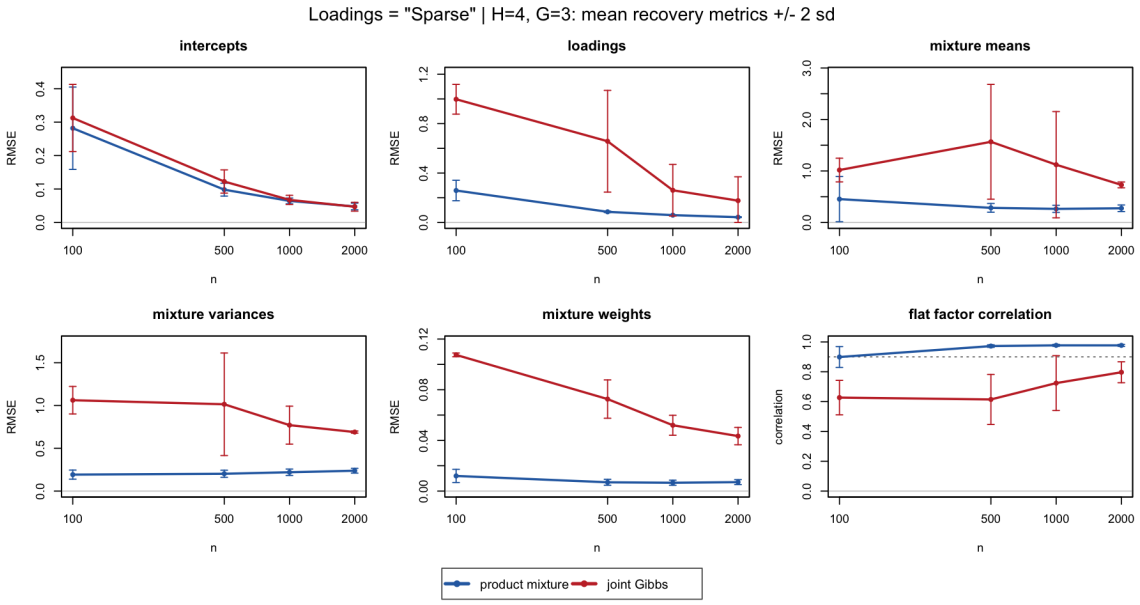



Figure 2: Parameter recovery in the $H = 4, G = 3$ setting with Sparse loadings. Lower is better for RMSE panels; higher is better for factor correlation.

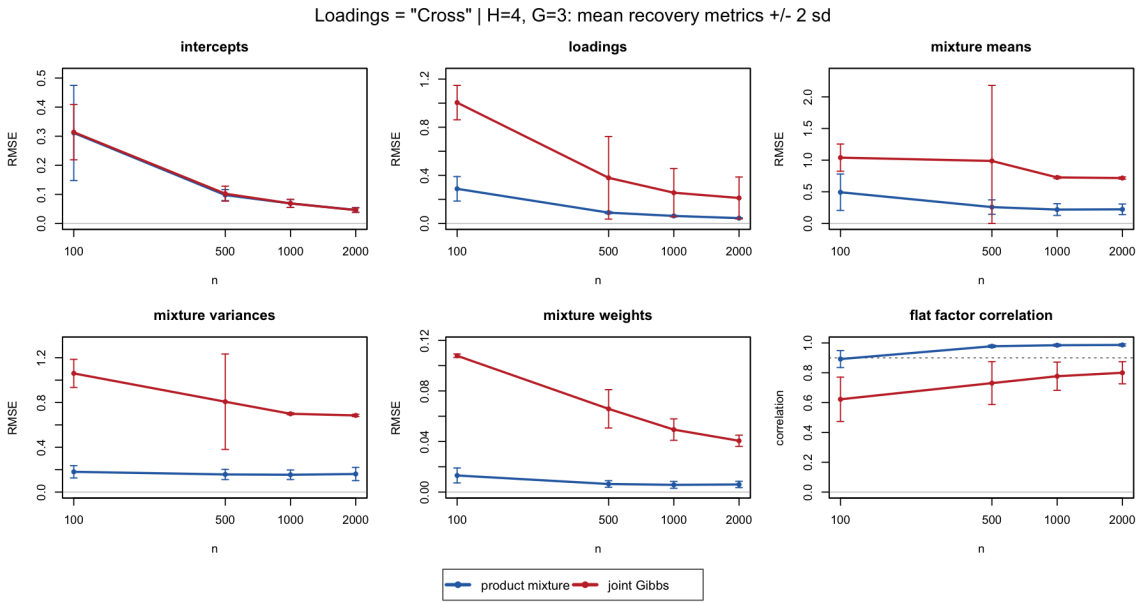
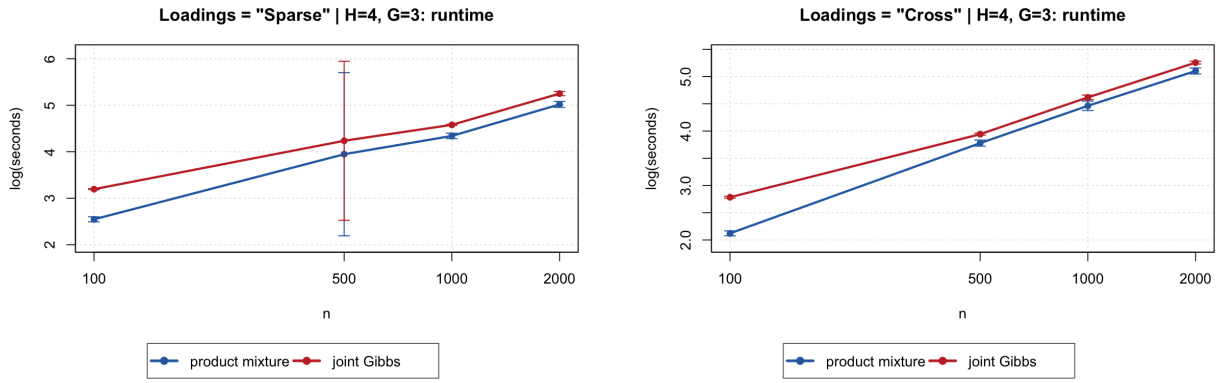



Figure 3: Parameter recovery in the $H = 4, G = 3$ setting with Cross loadings. Lower is better for RMSE panels; higher is better for factor correlation.




(a) Sparse loadings.

(b) Cross loadings.

Figure 4: Runtime summaries in the $H = 4, G = 3$ setting. The y-axis is log(seconds).

# 6 Reproducibility

The main simulation is launched with:

```
Rscript scripts/sample_size/run_moderate_crossloading_sample_size_MAP_intercepts.R
```

The paper-facing figures are regenerated from the completed checkpoint by setting `OUT_DIR` to the full sample-size output directory and running:

```
Rscript scripts/sample_size/plot_dgp_loading_heatmaps.R
Rscript scripts/sample_size/plot_sample_size_rmse_panels.R
Rscript scripts/sample_size/plot_sample_size_timing_lines.R
```

The selected summary tables used to audit these results are:

- `results/selected_tables/sample_size/checkpoint_parameter_recovery_summary.csv`;
- `results/selected_tables/sample_size/checkpoint_timing_log_seconds_summary.csv`;
- `results/selected_tables/sample_size/dgp_lambda_matrix_*.csv`.